Deployment guide

How to stand up your own copy of Savant Azurian in Azure, from an empty subscription to a working agent, with Bicep and the Azure Developer CLI (`azd`). Everything personal (names, IDs, URLs) comes from your own `azd` environment: nothing personal is stored in the repository. Every variable is described in the configuration reference; the reasoning behind the choices is in ADR-0023. A PDF version is generated from this page (`just docs-pdf`).

> **Status: not yet validated against a real subscription, and the Bicep was not compiled.** `infra/` was written from the Azure and Foundry documentation. The Bicep compiler could not be installed in the environment where this was written, so `bicep build` has never run. Treat the first deployment as a validation: start with `azd provision --preview`, and fix what the compiler or the preview reports. The points most likely to need a fix are in What to verify first. Remove this note after a clean deployment.

The short version

```
azd auth login
azd env new savant-dev
azd env set AZURE_LOCATION <region>
azd env set AZURE_SUBSCRIPTION_ID <subscription-id>
azd env set NOTION_OWNER_ID "$(python3 -c 'import uuid; print(uuid.uuid4())')"
azd env set BFF_OWNER_OBJECT_ID "$(az ad signed-in-user show --query id -o tsv)"

azd provision --preview && azd provision        # resources and most roles
scripts/setup/generate-notion-key.sh --apply     # the key goes straight to Key Vault
azd env set BFF_ENCRYPTION_KEY_READY true && azd provision
azd deploy sa2-agent                             # the hosted agent
# then: agent roles, backend image, Easy Auth, agent URL. See the steps below.
```

Plan on about an hour the first time, most of it waiting for Azure and doing the Easy Auth portal steps.

What gets created

One resource group, `rg-<environment>`, with:

Resource	Purpose	Notes
Foundry (AI Services) account and project	Hosts the agent; serves the models and Azure Speech	Entra ID only, keys disabled
Chat and embedding model deployments	Conversation; document vectorization	Names, model and capacity are parameters
Azure AI Search	Per-project document index	Entra ID only; the index is created by `scripts/search_setup.py`
Cosmos DB for NoSQL (serverless)	Projects, sessions, messages	Entra ID only
Storage account	Encrypted Notion tokens, project documents	Shared keys disabled
Key Vault	Notion encryption key, optional SSH key	RBAC authorization
Container Registry	Backend image	Admin user disabled
Container Apps environment and app	The backend (BFF)	Starts on a placeholder image
Log Analytics and Application Insights	Logs and traces	Connected to the Foundry project
Two managed identities	Backend runtime; GitHub Actions (OIDC)	No password anywhere

Who does what

Task	Done by	Why
Resource group, resources, model deployments, Cosmos roles, most role assignments	**Bicep** (`infra/`)	Declarative and repeatable
Notion encryption key, SSH key, GitHub variables, agent roles	**Scripts** (`scripts/setup/`)	Need a secret value, or an identity that does not exist before the first deployment
Search index, indexer, skillset	`scripts/search_setup.py`	Search data plane, not ARM
Hosted agent	`azd deploy`	Foundry data plane
Toolbox (web search, code interpreter)	`azd ai toolbox` (toolbox guide)	Foundry data plane
Easy Auth app registration	**You**, in the portal	Creates a directory object and a client secret
Notion consent	**You**, in the web interface	The owner's authorization is the point

Every script in `scripts/setup/` is a **dry run** until you add `--apply`: it prints the commands and changes nothing.

Prerequisites

- An Azure subscription where you can create resources **and** role assignments (Owner, or Contributor plus User Access Administrator).
- A region where your chat model and hosted agents are available. Check the model and quota first: `MODEL_NAME`, `MODEL_VERSION`, `MODEL_SKU` and `MODEL_CAPACITY` default to the values the project was built with.
- `azd` (developed with 1.37.3) with the `microsoft.foundry` extension, `az`, `gh`, Python 3.13 with `uv`.
- A Notion account for the Notion tools.
- The dev container installs all of these for you.

Step by step

1. Create the environment

```
azd auth login
azd env new <name>                              # for example: savant-dev
azd env set AZURE_LOCATION <region>
azd env set AZURE_SUBSCRIPTION_ID <subscription-id>
azd env set NOTION_OWNER_ID "$(python3 -c 'import uuid; print(uuid.uuid4())')"
azd env set BFF_OWNER_OBJECT_ID "$(az ad signed-in-user show --query id -o tsv)"
azd env set OIDC_GITHUB_REPOSITORY <owner>/<repo>      # optional, for GitHub
Actions
```

`NOTION_OWNER_ID` is a fixed key of your choosing, not a Notion identifier: any stable string, the same on the agent and the backend. `BFF_OWNER_OBJECT_ID` is your Entra object ID, which the backend compares with the caller's.

Optional model settings are described in the configuration reference.

2. Preview, then provision

```
azd provision --preview     # shows what would be created, changes nothing
azd provision               # creates it; takes several minutes
```

Outputs such as `KEY_VAULT_NAME`, `SEARCH_ENDPOINT` and `BFF_URL` are now in the azd environment (`azd env get-values`).

3. Create the Notion encryption key

```
scripts/setup/generate-notion-key.sh            # dry run
scripts/setup/generate-notion-key.sh --apply
azd env set BFF_ENCRYPTION_KEY_READY true
azd provision                                   # gives the backend the key by
secret reference
```

The key is generated straight into Key Vault and never shown. It is not overwritten if it exists: rotating it would make stored Notion tokens unreadable. The backend image has no Key Vault SDK: Container Apps reads the secret with the backend's identity and injects it, which is why the second `azd provision` waits until the key exists.

4. Create the document index

Search needs your own identity (the deployer roles come from step 2). Put the non-secret values in `.env`, then:

```
azd env get-values | grep -E '^(SEARCH_|AZURE_AI_PROJECT_ENDPOINT)' >> .env
uv run --locked python -m scripts.search_setup            # show the definitions
uv run --locked python -m scripts.search_setup --apply
```

Details: search documents.

5. Create the toolbox (optional)

Create the `sa2-web` toolbox (web search, code interpreter) as described in the toolbox guide. `TOOLBOX_ENDPOINT` already points at it. To run without a toolbox: `azd env set TOOLBOX_ENDPOINT ""`.

6. Deploy the agent

```
azd deploy sa2-agent
```

This creates the hosted agent and, with it, **the agent's own identity**.

7. Grant the agent's roles

Find the agent identity's object ID and client ID (Foundry portal, the agent's identity; or `azd ai agent show`), then:

```
scripts/setup/grant-agent-roles.sh --agent-principal-id <object-id>           # dry run
scripts/setup/grant-agent-roles.sh --agent-principal-id <object-id> --apply
azd env set NOTION_OAUTH_MANAGED_IDENTITY_CLIENT_ID <client-id>
azd deploy sa2-agent        # picks up the client ID
```

The script also gives the backend identity the role to call the agent.

8. Publish the backend image and roll it out

The app starts on a placeholder image. Build the real one in the cloud, from the public repository, so nothing from your machine is uploaded:

```
REGISTRY="$(azd env get-value ACR_NAME)"
az acr build --registry "$REGISTRY" --image sa2-bff:first --file Dockerfile.bff \
  https://github.com/<owner>/<repo>.git#main
IMAGE="$(azd env get-value ACR_LOGIN_SERVER)/sa2-bff:first"
az containerapp update --name "$(azd env get-value BFF_APP_NAME)" \
  --resource-group "$(azd env get-value AZURE_RESOURCE_GROUP)" --image "$IMAGE"
azd env set BFF_IMAGE "$IMAGE"
```

Use your own repository URL and branch if you forked. Do **not** run `az acr build .` from a local checkout: the registry build would upload everything in the folder, including `.env`, `.azure/` and `.venv`, because it ignores the BuildKit ignore file this repository uses. If you must build from local changes, use `docker build -f Dockerfile.bff` and push the image yourself.

Recording `BFF_IMAGE` matters: `azd provision` re-applies the template, and without it the app would go back to the placeholder. After that, the *Publish BFF* workflow publishes images and rolling one out stays a deliberate step.

9. Turn on Easy Auth (Azure portal)

The backend only accepts the owner. Easy Auth validates the Microsoft Entra token before the backend sees it.

1. Container App → *Authentication* → *Add identity provider* → *Microsoft* → *Create new app registration* (single tenant). The portal creates the registration, the `api://<client-id>` URI, the `user_impersonation` scope and a client secret kept as a Container App secret (valid 365 days: renew it).
2. *Restrict access*: **Allow unauthenticated access**, otherwise `/readiness` and the Notion callback are blocked. The backend answers `401` itself when no identity header exists.
3. *Client application requirement*: **specific client applications**, with the Azure CLI application ID `04b07795-8ddb-461a-bbee-02f9e1bf7b46`. *Identity requirement*: any identity.
4. App registration → *Expose an API* → *Authorized client applications*: add the Azure CLI ID with `user_impersonation` checked. Without it, `az account get-access-token` fails with `AADSTS65001`.
5. Sign in with `az login` in the browser. Security Defaults block the device-code flow.

Then verify that a call without a token returns `401` and your own token returns `200` on `/api/oauth/notion/status`. The identity model and the `curl` checks are in the backend API notes; the decision is in ADR-0011.

10. Connect the backend to the agent, last

Only now, set the agent endpoint and re-provision:

```
azd env set AGENT_RESPONSES_URL "<agent /responses URL>"
azd provision
```

The template adds `AGENT_RESPONSES_URL` to the backend only when `NOTION_OWNER_ID` and `BFF_OWNER_OBJECT_ID` are also set, so the app cannot start half configured.

11. GitHub Actions (optional)

```
scripts/setup/github-variables.sh --repo <owner>/<name>            # dry run
scripts/setup/github-variables.sh --repo <owner>/<name> --apply
```

GitHub signs in to Azure through OpenID Connect with the CI identity, so no Azure secret is ever stored. `CD` and `Publish BFF` deploy as soon as CI is green on `main`; keep them disabled until you want that.

12. Use it

Run the web interface locally (web client guide) and connect Notion from *Settings*. The user guide walks through the interface.

Roles

Identity	Role	On	Granted by
Backend	Key Vault Secrets User	vault	Bicep
Backend	Storage Blob Data Contributor	`notion-oauth` and `documents` containers	Bicep
Backend	Search Index Data Contributor, Search Service Contributor	search service	Bicep
Backend	Cognitive Services Speech Contributor	Foundry account	Bicep
Backend	AcrPull	registry	Bicep
Backend	Cosmos DB Built-in Data Contributor (data plane)	Cosmos account	Bicep
Backend	Foundry Agent Consumer	Foundry project	`grant-agent-roles.sh`
Search service	Storage Blob Data Reader	storage account	Bicep
Search service	Cognitive Services OpenAI User	Foundry account	Bicep
CI (OIDC)	Foundry User	Foundry account	Bicep
CI (OIDC)	AcrPush, Reader	registry	Bicep
You (deployer)	Key Vault Secrets Officer, Foundry User, Search Service Contributor, Search Index Data Contributor	vault, Foundry, search	Bicep
Hosted agent	Key Vault Secrets User	vault	`grant-agent-roles.sh`
Hosted agent	Storage Blob Data Contributor	`notion-oauth` container	`grant-agent-roles.sh`
Hosted agent	Search Index Data Reader	search service	`grant-agent-roles.sh`

Identity	Role	On	Granted by
Hosted agent	Cognitive Services OpenAI User	Foundry account	`grant-agent-roles.sh`
Hosted agent	Monitoring Metrics Publisher	Application Insights	`grant-agent-roles.sh`

The Speech role is **Contributor**, not User: User cannot call `issuetoken`, which the backend uses to hand the browser a 10-minute token.

Troubleshooting

Symptom	Likely cause	What to do
`azd provision` fails on a model deployment	The model, version or SKU is not offered in the region, or there is no quota	Pick another region, or set `MODEL_NAME`, `MODEL_VERSION`, `MODEL_SKU`, `MODEL_CAPACITY`; check the quota in the Foundry portal
`azd provision` fails on a role assignment	You lack `Microsoft.Authorization/roleAssignments/write`	Use Owner, or Contributor plus User Access Administrator
The second `azd provision` fails on the Container App secret	The Key Vault secret does not exist yet, or the backend role is not propagated	Run the key script with `--apply`; wait a few minutes; run it again
The backend answers `401` on every `/api` call	Easy Auth is not set up, or the token audience is wrong	Redo step 9; get a token with `az account get-access-token --resource api://<client-id>`
The backend answers `403`	Your object ID differs from `BFF_OWNER_OBJECT_ID`	Set it to the output of `az ad signed-in-user show --query id -o tsv` and re-provision
The backend does not start	`AGENT_RESPONSES_URL` is set without `NOTION_OWNER_ID` and `BFF_OWNER_OBJECT_ID`	Set all three; the template only sets the URL when the other two exist

Symptom	Likely cause	What to do
The agent cannot read the Notion store or the vault	Its roles are missing (it has its own identity)	Run `grant-agent-roles.sh` with the agent's object ID
`AADSTS65001` or `AADSTS650057` when getting a token	The Azure CLI is not pre-authorized on the API	Step 9, item 4
`AADSTS530035`	Security Defaults block the device-code flow	Use `az login` in the browser
The app is back on the placeholder image after `azd provision`	`BFF_IMAGE` was not recorded	`azd env set BFF_IMAGE <image>` and provision again
A name is "already taken" after `azd down`	A soft-deleted Key Vault or AI Services account still holds it	`azd down --purge`, or purge it in the portal

What it costs

Rough monthly order of magnitude for a single owner with light use. **Check the Azure pricing calculator for your region**: prices change and these figures are estimates, not quotes.

Resource	Cost pattern	Order of magnitude
Azure AI Search, Basic	Fixed, billed while it exists	The largest fixed cost, roughly US$70 to 80
Container Registry, Basic	Fixed	A few US$
Container Apps (scale to zero)	Per use	Near zero when idle
Cosmos DB serverless	Per request and storage	Cents to a few US$
Storage, Key Vault	Per use	Cents
Log Analytics, Application Insights	Per GB ingested	Free allowance, then cents to a few US$
Models and Speech	Per token and per audio minute	Depends entirely on usage

Tearing it down

```
azd down --purge
```

This deletes the resource group and purges the soft-deleted Key Vault and AI Services account so their names can be reused. It removes the Notion token store and the documents with it. The Easy Auth app registration is a directory object outside the resource group: delete it in Entra if you no longer need it.

What to verify first

This template has not met a real subscription yet. On the first run, check:

1. `bicep build` **and** `azd provision --preview` compile and list what you expect. Fix any API-version or property error they report.
2. `azd` **with** `azure.yaml`**.** The `ai-project` service uses `host: azure.ai.project` with an `endpoint` from the environment, next to an `infra:` block. Confirm `azd provision` and `azd deploy` accept that combination, and that `${VAR}` substitution works for the `endpoint` field and for variables left empty (the SSH ones).
3. **Hosted agent prerequisites.** Check with the Foundry documentation whether a hosted agent needs anything the template does not create (for example a capability host or a registry connection). A project created in the portal worked with the same account and project resources.
4. **Region and quota** for the chat model, embedding model and hosted agents.

5. **The Application Insights connection** on the Foundry project (`appinsights`): confirm agent telemetry reaches Application Insights.
6. **Role names.** `Foundry Agent Consumer` is looked up by name in `grant-agent-roles.sh`; confirm it exists in your tenant.
7. **The Container App secret reference** to Key Vault in `infra/modules/app.bicep`: the API shape (`keyVaultUrl` and `identity` under `secrets`) follows the Container Apps documentation but has not been deployed.

Related

- Configuration reference: every variable and secret.
- Toolbox, search documents, SSH tool, workspace on Cosmos DB.
- Component delivery: what a merge to `main` deploys.
- ADR-0020 and ADR-0023: why it is built this way.